

B+ Tree

B+树

- 1.索引字段值重复出现于叶结点和非叶结点；
- 2.指向主文件的指针仅出现于叶结点；
- 3.所有叶结点即可覆盖所有键值的索引；
- 4.索引字段值在叶结点中是按顺序排列的。

B树

- 1.索引字段值仅出现一次：在叶结点或非叶结点中；
- 2.指向主文件的指针出现于叶结点或非叶结点；
- 3.所有结点才能覆盖所有键值的索引。

Number of pointers/keys for B ⁺ -Tree				
	Max ptrs	Max keys	Min ptrs	Min keys
Non-leaf (non-root)	$n + 1$	n	$\lceil \frac{n+1}{2} \rceil$	$\lceil \frac{n+1}{2} \rceil - 1$
Leaf (non-root)	$n + 1$	n	$\lfloor \frac{n+1}{2} \rfloor + 1$	$\lfloor \frac{n+1}{2} \rfloor$
Root	$n + 1$	n	2^*	1

*When there is only one record in the B⁺-Tree, min pointers in the root is 1 (the other pointers are null)

1. What are the minimum numbers of keys and pointers in Btree (i) interior nodes and (ii) leaves, when:

a) $n = 10$; i.e., a block holds 10 keys and 11 pointers.

b) $n = 11$; i.e., a block holds 11 keys and 12 pointers.

Solution

interior nodes do contain $\lceil (n+1)/2 \rceil$ pointers while leaf contains $\lceil (n+1)/2 \rceil$ pointers to data.

The minimum key for interior node contains $\lceil (n+1)/2 \rceil - 1$ keys while leaf contains $\lceil (n+1)/2 \rceil$ keys

a) When $n = 10$:

interior node holds at least 5 keys and 6 pointers,

Leaves are at least 5 keys and pointer when extra pointer to next leaf block is not counted.

b) $n = 11$:

interior nodes contains at least 5 keys and 6 pointers,

Leaves contain at least 6 keys and pointers.

2. Suppose pointers are 4 bytes long, and keys are 12 bytes long. How many keys and pointers will a block of 16,384 bytes have?

Solution

Number of keys = 1023

Number of Pointers = $1023 + 1 = 1024$

Step-by-step explanation:

As we know the structure of B Tree, that each block will have n keys and $n+1$ pointers value .

n will be large to allow $n+1$ pointers and n keys

Therefore this means, $12n + 4(n+1) < 16384$, which implies that $12n + 4n + 4 < 16384$, which implies that $16n < 16380$, $n = 1023.75$

As said above that n will be less than equal, that means $n = 1023$.

Number of keys = 1023

Number of Pointers = $1023 + 1 = 1024$

Question 1

Assume that you have the following table:

Item		
SSN	name	age
1	Pete	13
2	Bob	23
44	John	49
43	Joe	45
45	Alice	77
42	Lily	3
88	Gertrud	29
89	Heinz	14

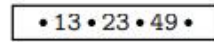
Create a B+-tree for table *Item* on key age with $n = 3$ (up to three keys per node). You should start with an empty B+-tree and insert the keys in the order shown in the table above. Write down the resulting B+-tree after each step.

When splitting or merging nodes follow these conventions:

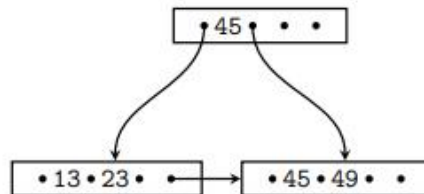
- **Leaf Split:** In case a leaf node needs to be split during insertion and n is even, the left node should get the extra key. E.g, if $n = 2$ and we insert a key 4 into a node $[1,5]$, then the resulting nodes should be $[1,4]$ and $[5]$. For odd values of n we can always evenly split the keys between the two nodes. In both cases the value inserted into the parent is the smallest value of the right node.
- **Non-Leaf Split:** In case a non-leaf node needs to be split and n is odd, we cannot split the node evenly (one of the new nodes will have one more key). In this case the “middle” value inserted into the parent should be taken from the right node. E.g., if $n = 3$ and we have to split a non-leaf node $[1,3,4,5]$, the resulting nodes would be $[1,3]$ and $[5]$. The value inserted into the parent would be 4.
- **Node Underflow:** In case of a node underflow you should first try to redistribute values from a sibling and only if this fails merge the node with one of its siblings. Both approaches should prefer the left sibling. E.g., if we can borrow values from both the left and right sibling, you should borrow from the left one.

Solution

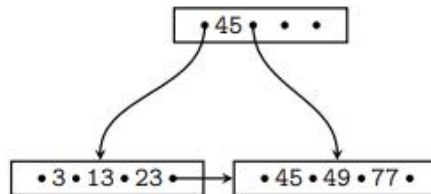
Insert 13, 23, and 49



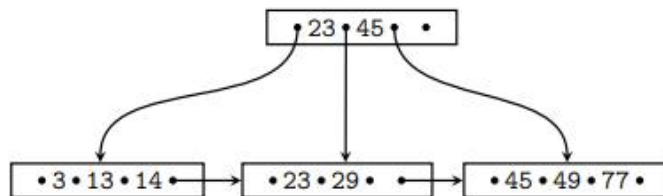
Insert 45. No room, node split



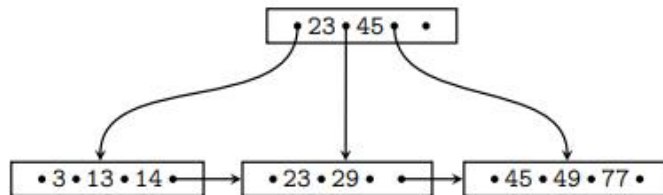
Insert 77 and 3



Insert 29. No room, node split



Insert 14

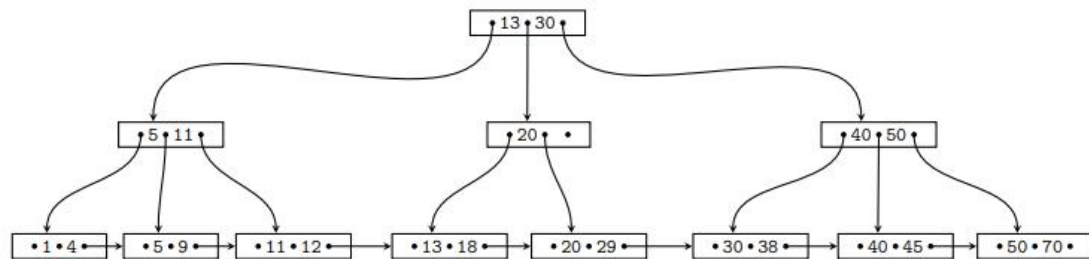


Question 2

Given is the B+-tree shown below ($n = 2$). Execute the following operations and write down the resulting B+-tree after each operation:

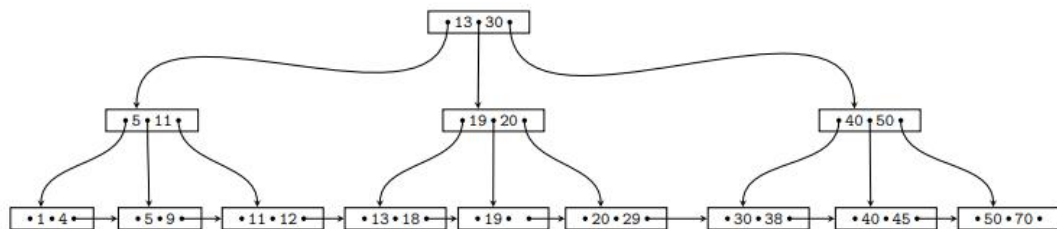
insert(19), insert(27), delete(5), delete(9), delete(12), delete(4), delete(11)

Use the conventions for splitting and merging introduced in the previous question.



Solution

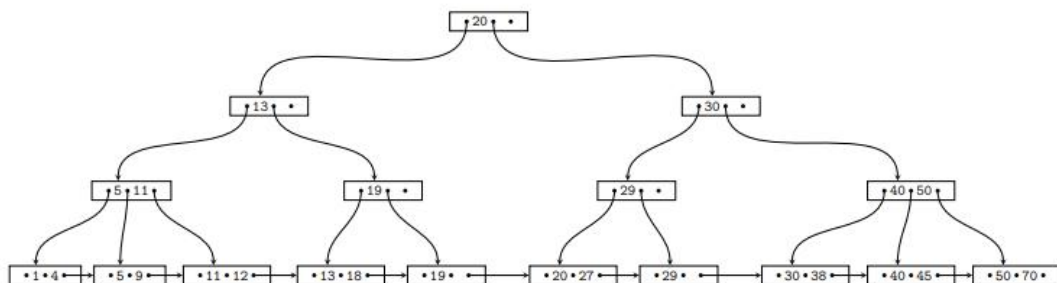
Insert 19



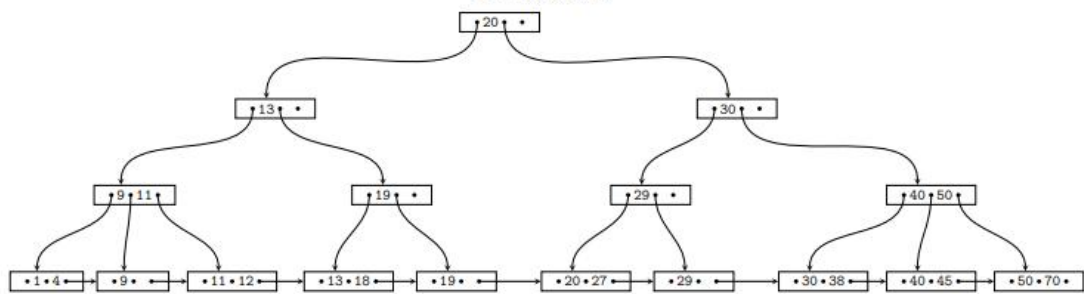
Insert 27

You to do the followin steps:

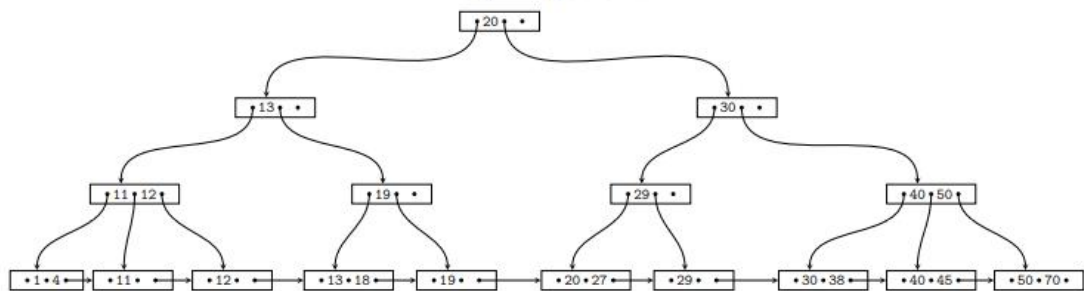
- split leaf; split parent;
- split grandparent
- create a new root with key 20



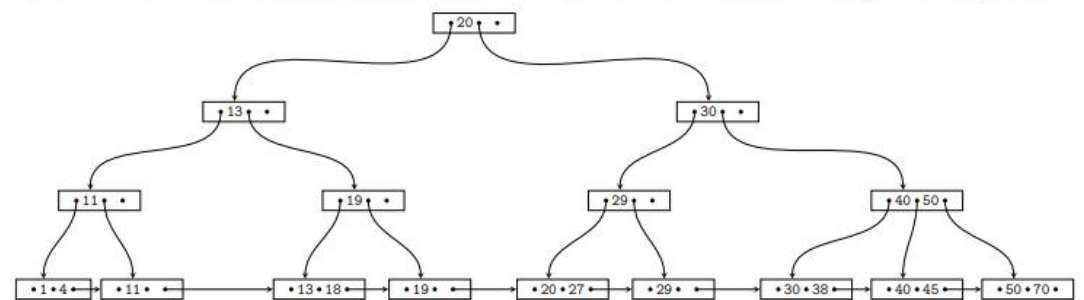
Delete 5



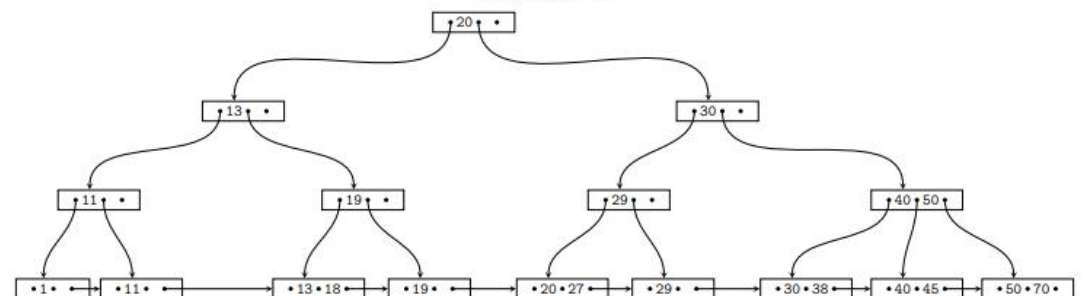
Delete 9: You need to redistribute from right sibling and update key in parent



Delete 12: You need to merge the leaves and delete key from parent



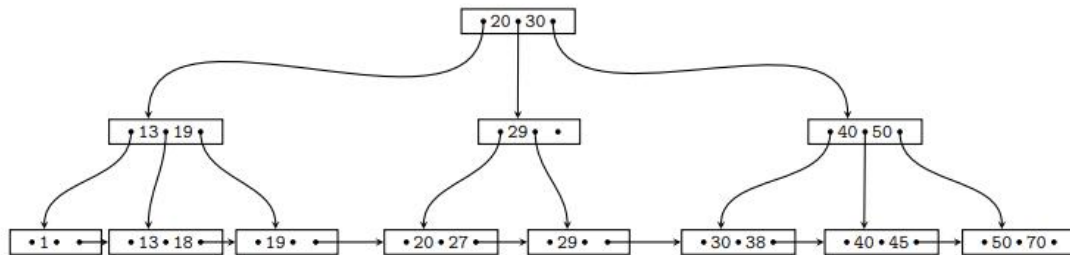
Delete 4



Delete 11:

You need to do the following:

- merge leaves; merge parent
- bring down key 13
- merge grandparent
- bring down key 20
- remove root



Question 3

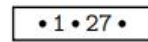
Assume that you have the following table:

Item		
id	name	price
1	Shovel	15
27	Sunflower seeds	3
44	Spate	23
7	Lawnmower	115
15	Lawnmower XL	497
20	Pine tree	256
32	Pine tree	19
3	Fertilizer	45

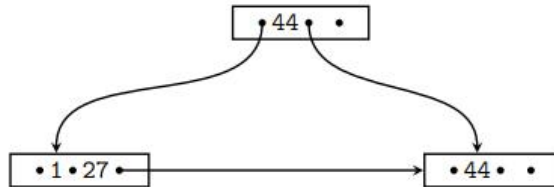
拆分合并节点的要求按照Question 1中的。

Solution

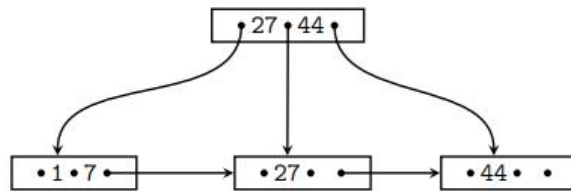
Insert 1, 27



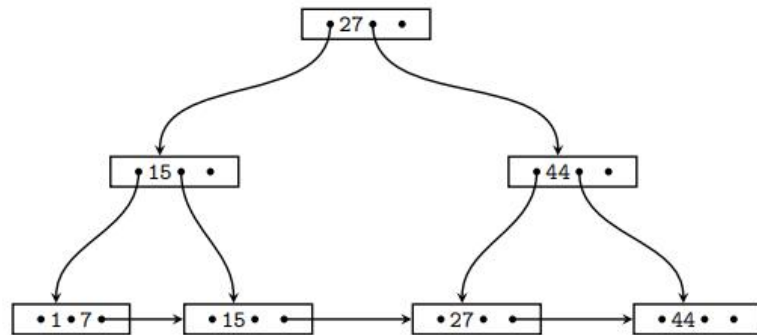
Insert 44



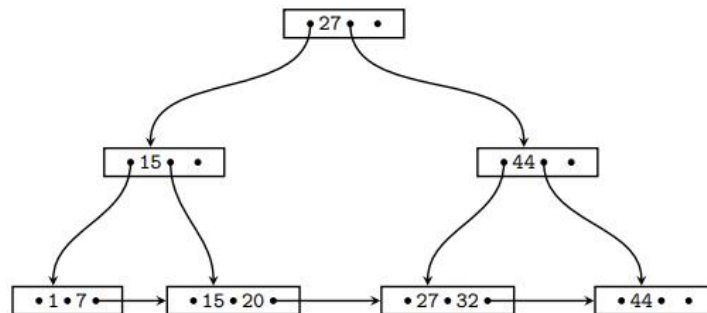
Insert 7



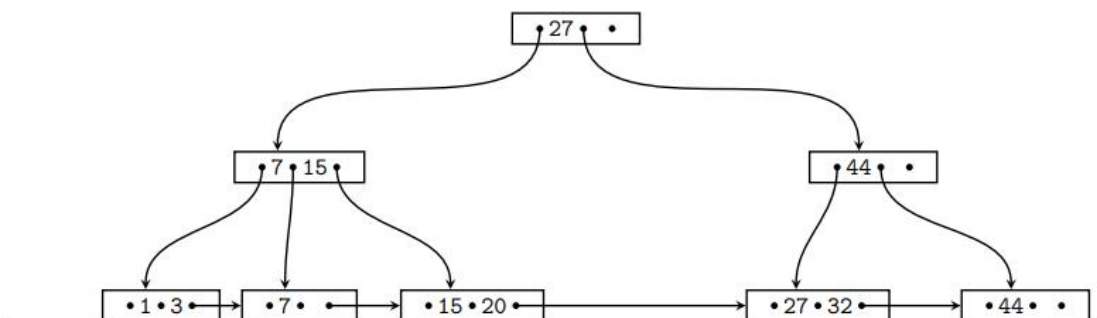
Insert 15



Insert 20, insert 32



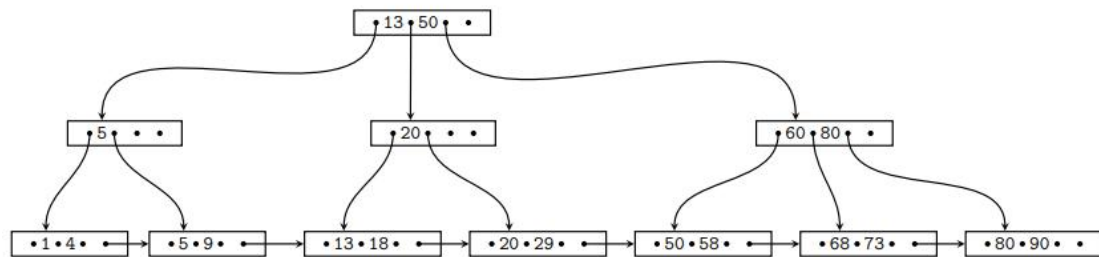
Insert 3



Question 4

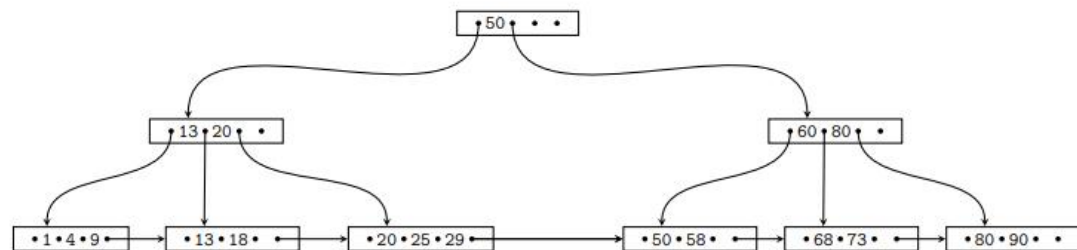
Given is the B+-tree shown below ($n = 3$). Execute the following operations and write down the resulting B+-tree after each operation:

delete(5), insert(25), delete(50), insert(28), delete(18), insert(35), delete(9) ,
Use the conventions for splitting and merging introduced in the previous question.

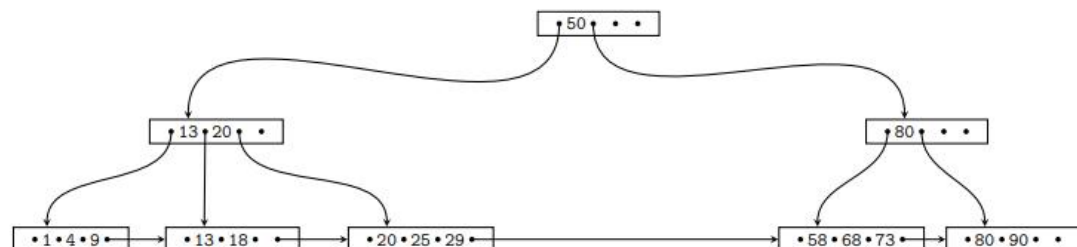


Solution

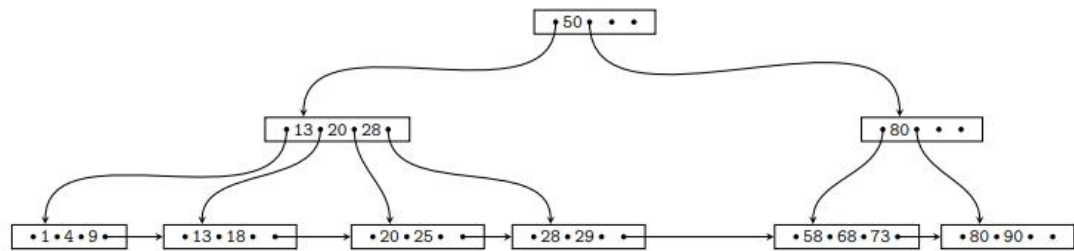
Delete 5, Insert 25



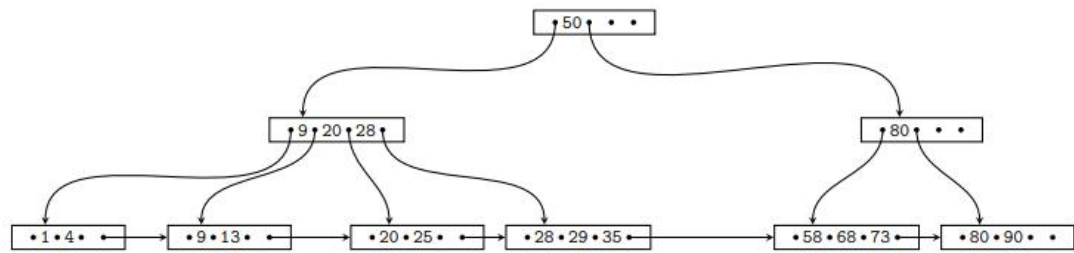
Delete 50



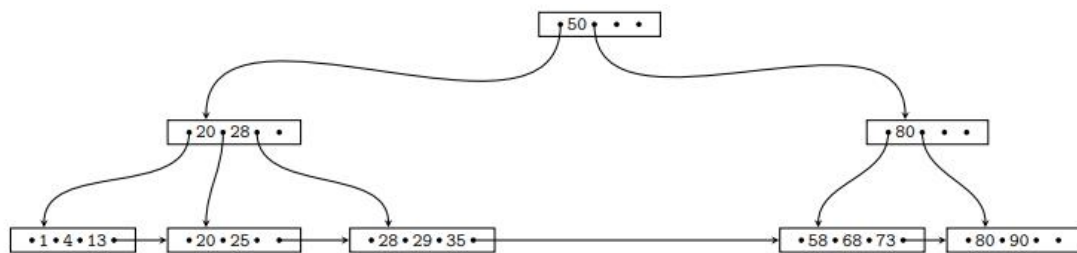
Insert 28



Delete 18, Insert 35

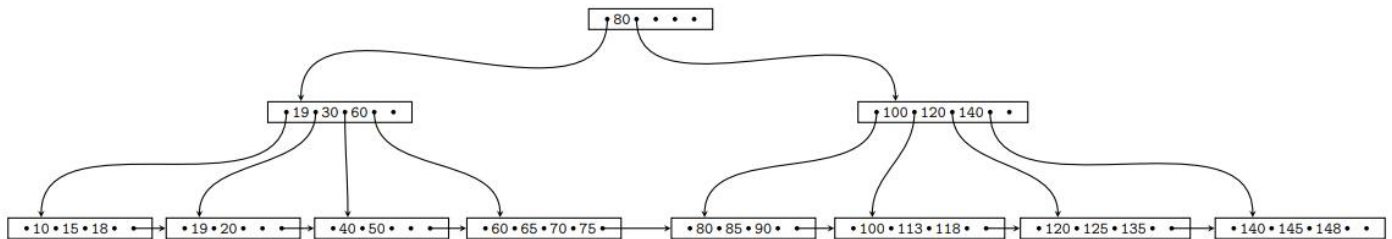


Delete 9



Question 5

Consider the following B+-tree index on the grade field. ($n = 4$).



1. What is the total number of records that can be inserted into this tree (more nodes may be created), while the height of the tree does not change.

Solution

Level 1 (Root): Can have up to 5 children (non-leaf nodes),

Level 2 (Non-Leaf nodes): each non-leaf node can have up to 5 children (leaf nodes).

Level 3 (Leaf nodes): Each leaf node can have up to 4 keys.

As a result, the total number of records can be inserted into this tree = $5 \times 5 \times 4 = 100$ records

Correction Guideline

2.5 Points for the correct # of records. Student must show his/her work, otherwise, 1 point.

2. For the following query, assume each record in the answer resides in a different page. Calculate the number of disk I/Os if we use this index to answer the following query. Briefly explain your calculation.

`SELECT * FROM R WHERE grade  $\geq$  19 AND grade  $\leq$  100;`

Solution

To answer the query, we need a total of 19 Disk I/Os

– 7 index page I/Os:

* Start from root, then non-leaf node [19, 30, 60], to leaf nodes [19, 20] till [100, 113, 118]

– 12 record page I/Os to fetch the records 19, 20, 40, 50, 60, 65, 70, 75, 80, 85, 90, 100

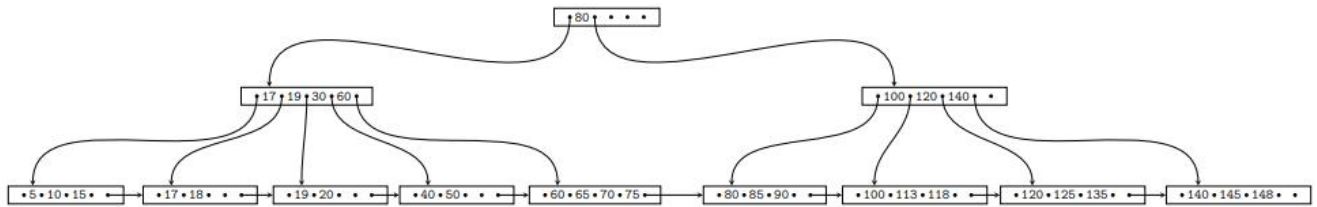
Correction Guideline

2.5 Points for the correct # of I/O. Student must show his/her work, otherwise, 1 point

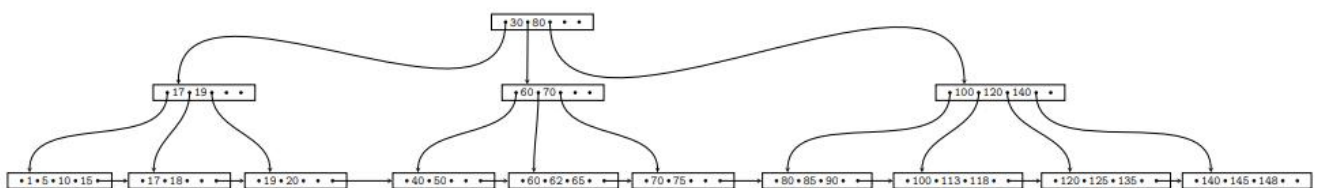
3. Draw the modified tree after inserting 5, 17, 62, and then 1, then deleting values 19, 80, 30, and then 120.

Solution

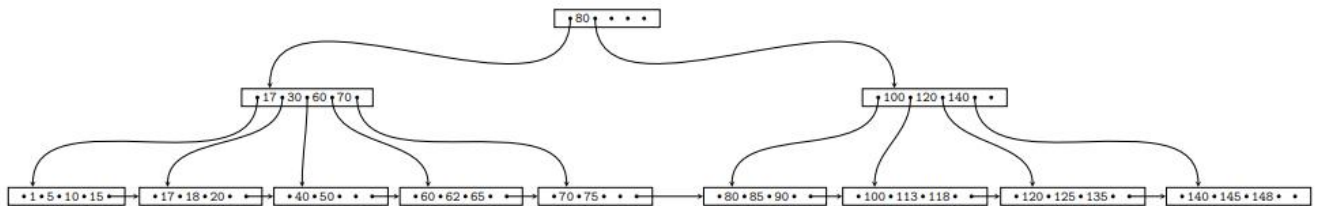
Insert 5, 17



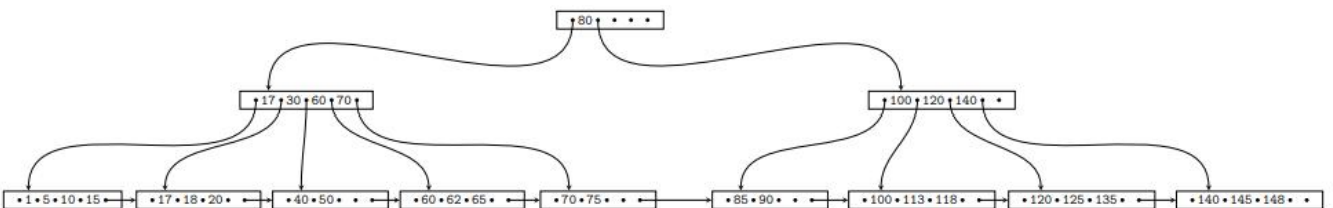
Insert 62, 1



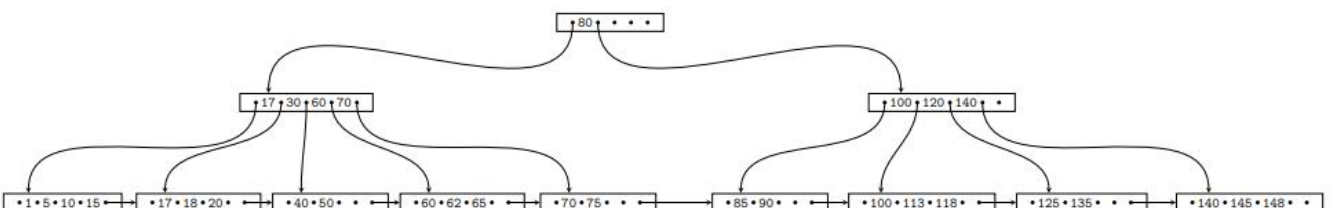
Delete 19



Delete 80, Delete 30 (Nothing Change)



Delete 120



Correction Guideline

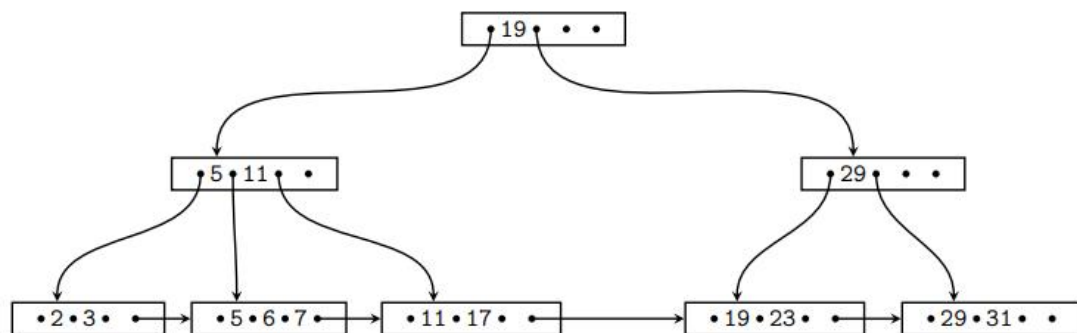
- 20 Points for correct solution
- Do not reduce points for missing leaf node pointers
- -5 Points for each incorrect leaf node
- -7 Points for each incorrect non-leaf node

Question 6

Given is the B+-tree shown below ($n = 3$). Execute the following operations and write down the resulting B+-tree after each operation:

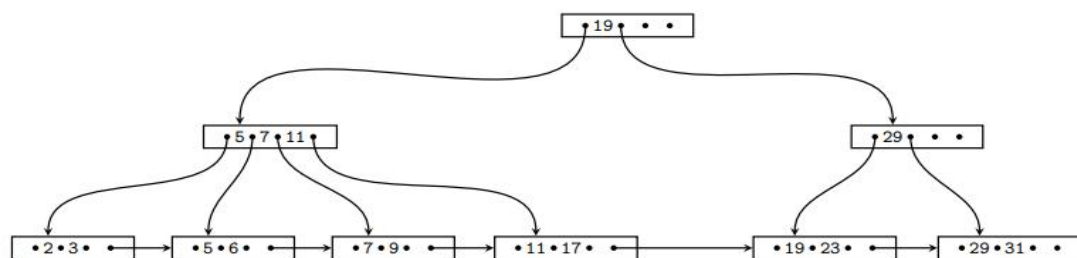
insert(9), delete(5), insert(10), insert(8), delete(23), delete(19), delete(11)

Use the conventions for splitting and merging introduced in the previous question.

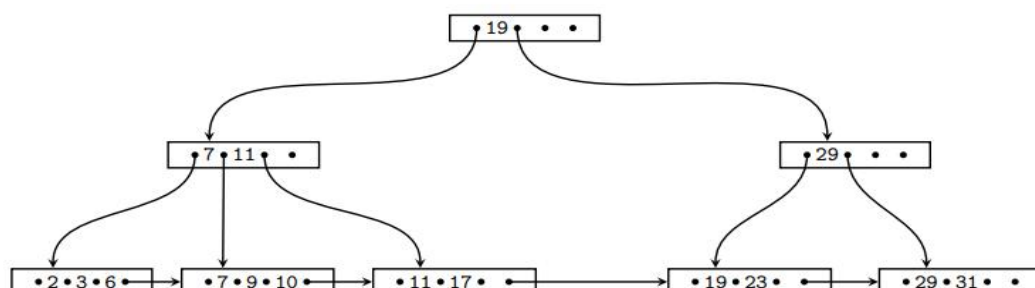


Solution

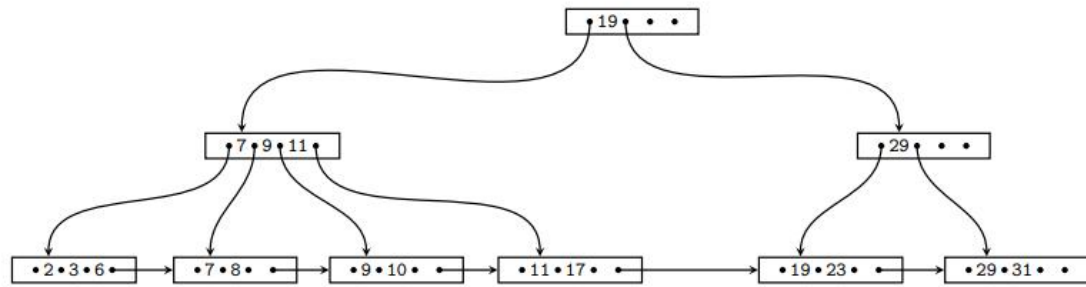
insert(9)



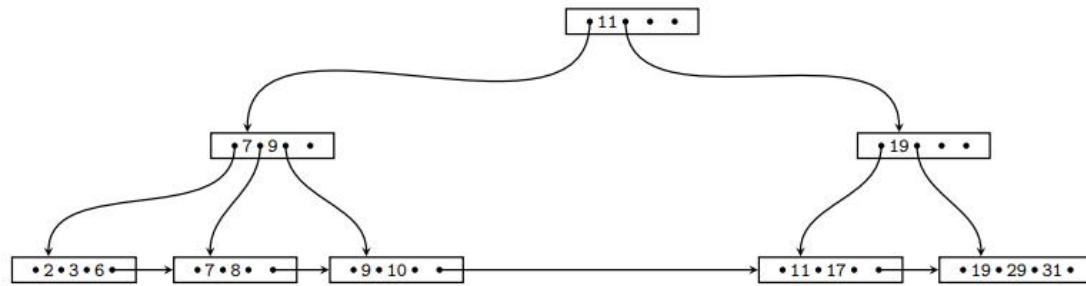
delete(5), then insert(10)



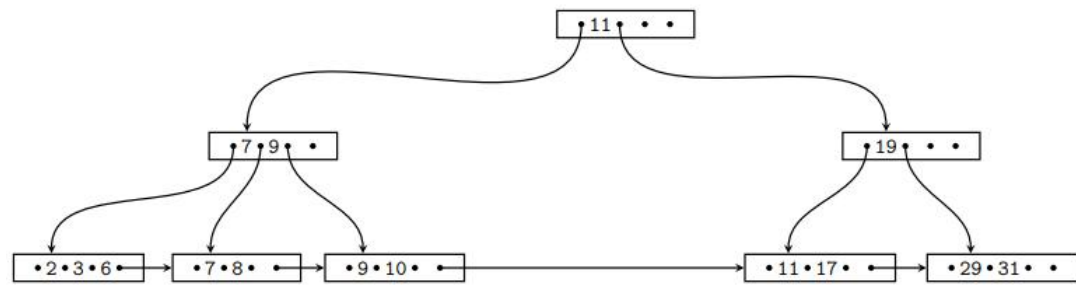
insert(8)



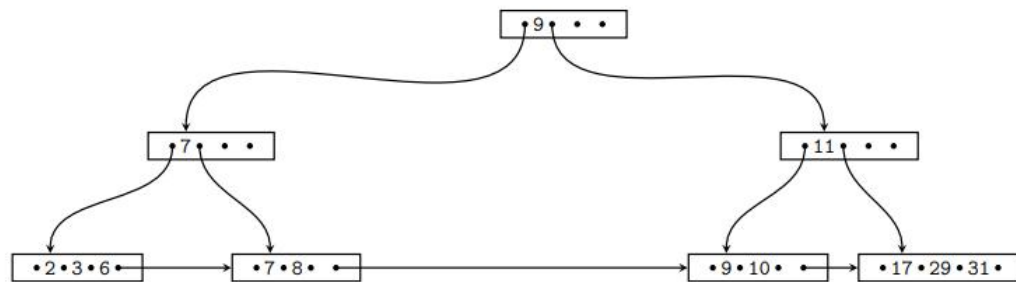
delete(23)



delete(19)



Steps to delete(11): remove [11] from leaf node, the node is underflow, Merge the underflow node [17] with right sibling, delete [left-pointer, 19] from the parent node. The parent node is underflow. Borrow/Transfer a key through the parent node. [9] is the new root, [11] is the new key in the parent node



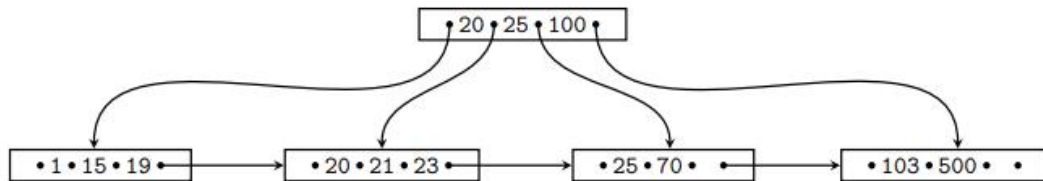
Question 7

Consider the B+-tree shown below ($n = 3$). Execute the following operations and write down the resulting B+-tree after each step:

insert(2),insert(3),insert(4),delete(103)

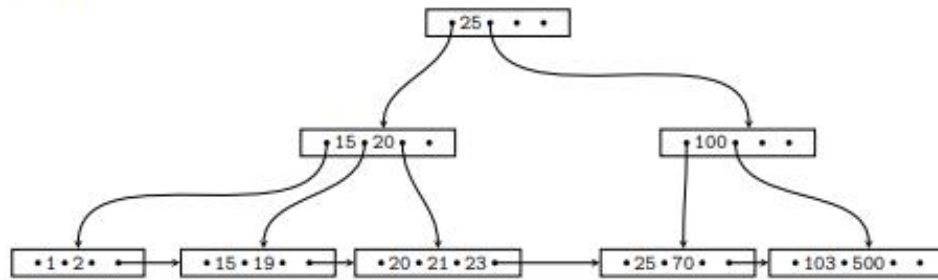
When splitting or merging nodes follow these conventions:

- **Leaf Split:** In case a leaf node needs to be split, the left node should get the extra key if the keys cannot be split evenly.
- **Non-Leaf Split:** In case a non-leaf node is split evenly, the “middle” value should be taken from the right node.
- **Node Underflow:** In case of a node underflow you should first try to redistribute and only if this fails merge. Both approaches should prefer the left sibling.

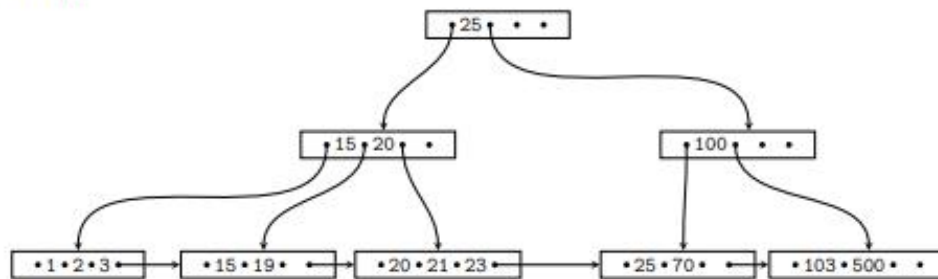


Solution

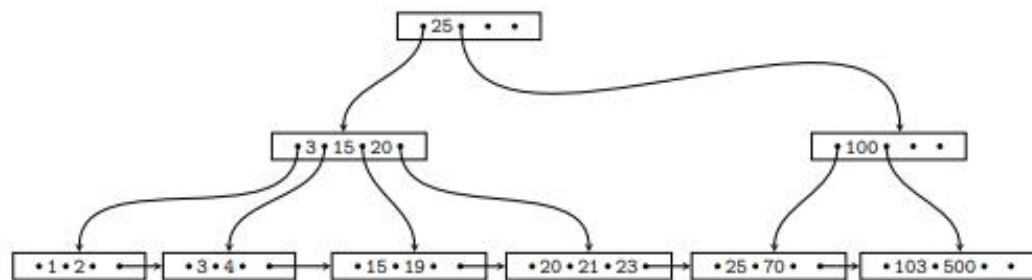
insert(2)



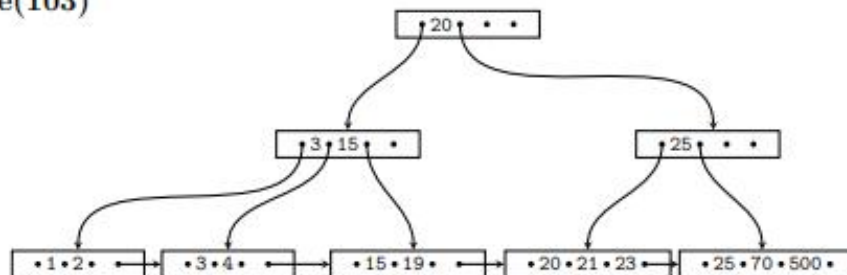
insert(3)



insert(4)



delete(103)



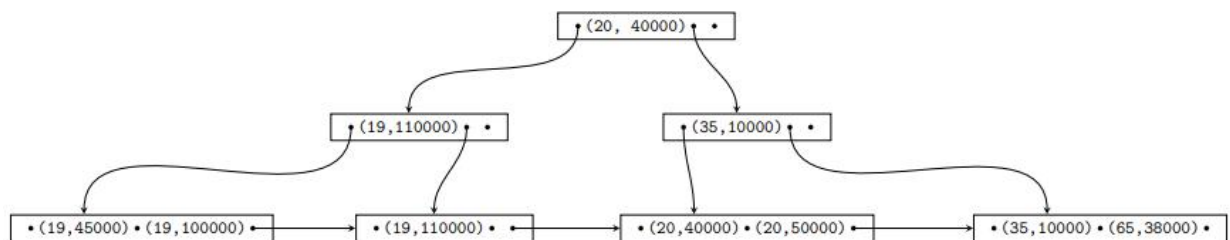
Question 8

Assume that you have the following table:

Item		
name	age	salary
Peter Petersen	35	10,000
Gert Gertsen	20	40,000
Heinz Heinzen	19	100,000
Gertrud Gertsen	19	45,000
Astrid Lundgren	19	110,000
Pferdegert	20	50,000
Heinz Bert	65	38,000

Create a B+-tree for table Item over attributes age and salary with $n = 2$ (up to two keys per node). You should start with an empty B+-tree and insert the keys in the order shown in the table above. Write down the resulting B+-tree after each step.

Solution

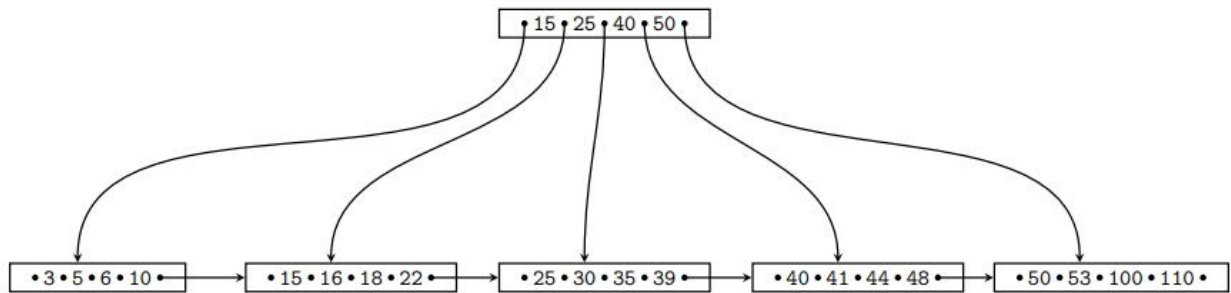


Question 9

Given is the B+-tree shown below ($n = 4$). Execute the following operations and write down the resulting B+-tree after each operation:

delete(15), delete(16), delete(18), insert(1), insert(2), insert(4), insert(200), insert(180)

Use the conventions for splitting and merging introduced in the previous question.



Solution

